String in C++

Tong Yi

Table of contents

- Motivation of Using String
- Motivation of Class
- Examples of Classes
- Primitive Type vs. Class and Objects
- Declare and Instantiate a String Object
- String is a Class
- Methods of String
  - length
  - concatenation operator +
  - index operator []
  - at
  - find
- Method Overload
  - find Method Example
  - substr

Motivation of Using String

- Perform calculations using numerical types like int and double. Input the radius of a circle, calculate and print out its area.

$$area = \pi(radius)^2$$

Complete code is onlinegdb link.

```
std::cout << "Enter the radius of a circle: ";
double radius;
std::cin >> radius;

double area = M_PI * radius * radius; //#include
    <cmath> for M_PI
std::cout << "area of a circle with radius "
   << radius << " is " << area << std::endl;
```

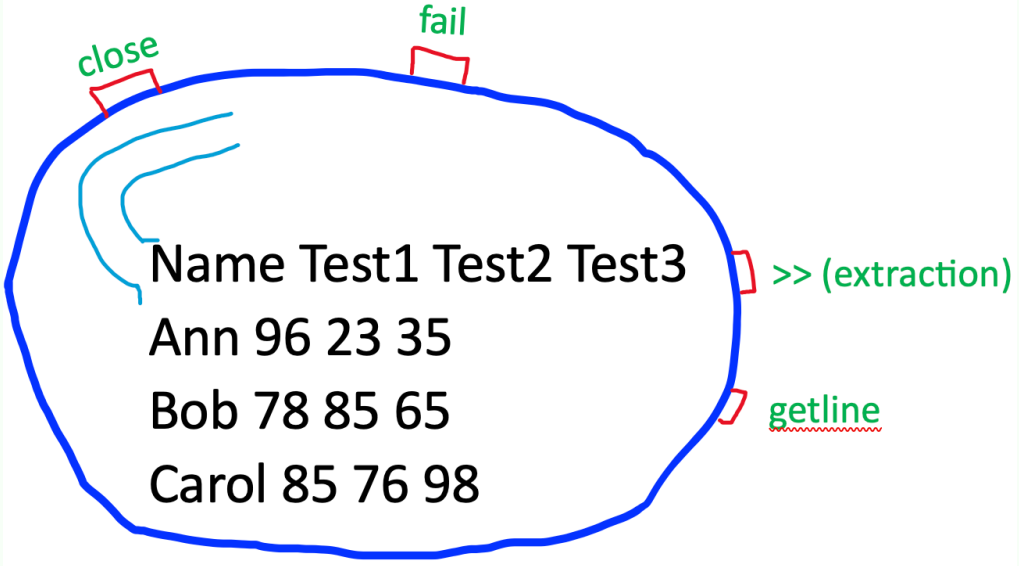
- We like symbols. How to represent a name in C++?
- A string is a sequence of characters, enclosed in double quotes.

Motivation of Class

- Different objects have different data members (aka, attributes).
  - *highest educational qualification* of a person? ✓
    highest educational qualification of a car? ✗
  - *full tank capacity* of a car? ✓
    full tank capcacity of a person? ✗
- Furthermore, different data have different operations.
  - multiply two integers, say, 11 * 12, is ok. ✓
    That is, mulplication is a valid operation for integers.
  - multiply two strings, say, `"Hello" * "world"`, is invalid ✗
    Or, multiplication is not a valid operations for strings.
- How about we enclose appropriate data (data members) of an object in a box, and provide only valid functions (aka methods or operations) for those data? Users can only access / modify data through the provided methods.
- class is **encapsulation** of **data** and **operations** on those data.

Examples of Classes

ifstream class to read text file



hypothetical intelligent hamburger

meat with cooking instruction

Primitive Type vs. Class

- Primitive type variables store values directly.

```
int i = 5;
```

Important: primitive type do not have built-in methods. No dot operator can follow such a variable. Say, i.numDigits() ✗

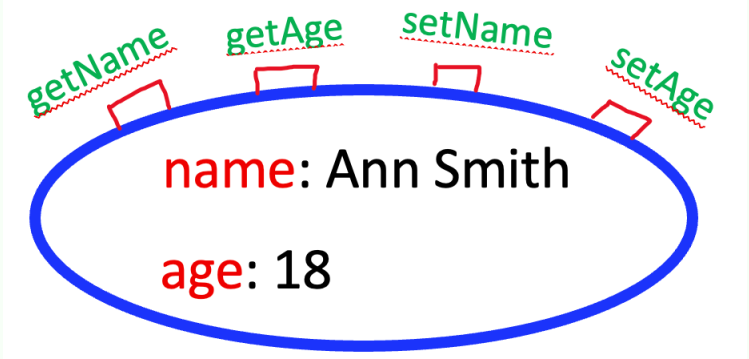
- class and object

A class is a blueprint that encapsulates both the kind of data the object will store and the methods that manipulate that data.

string name; int age;
getName getAge setName setAge

An object is a specific instance created from that blueprint.

ann is an object of Person class. Her name is `Ann Smith` and age is 18.



Declare and Instantiate a String Object

Declare and instantiate a string object `str` with empty string.

Declare and Instantiate a String Object

Declare and instantiate a string object `str` with empty string.

```
std::string str; //an empty string
```

Declare and instantiate a string object called `greeting` with value `"Hello"`.

Declare and Instantiate a String Object

Declare and instantiate a string object `str` with empty string.

```
std::string str; //an empty string
```

Declare and instantiate a string object called `greeting` with value `"Hello"`.

```
std::string greeting("Hello");
```

Unlike other classes, the String class can be instantiated as if it were a primitive type.

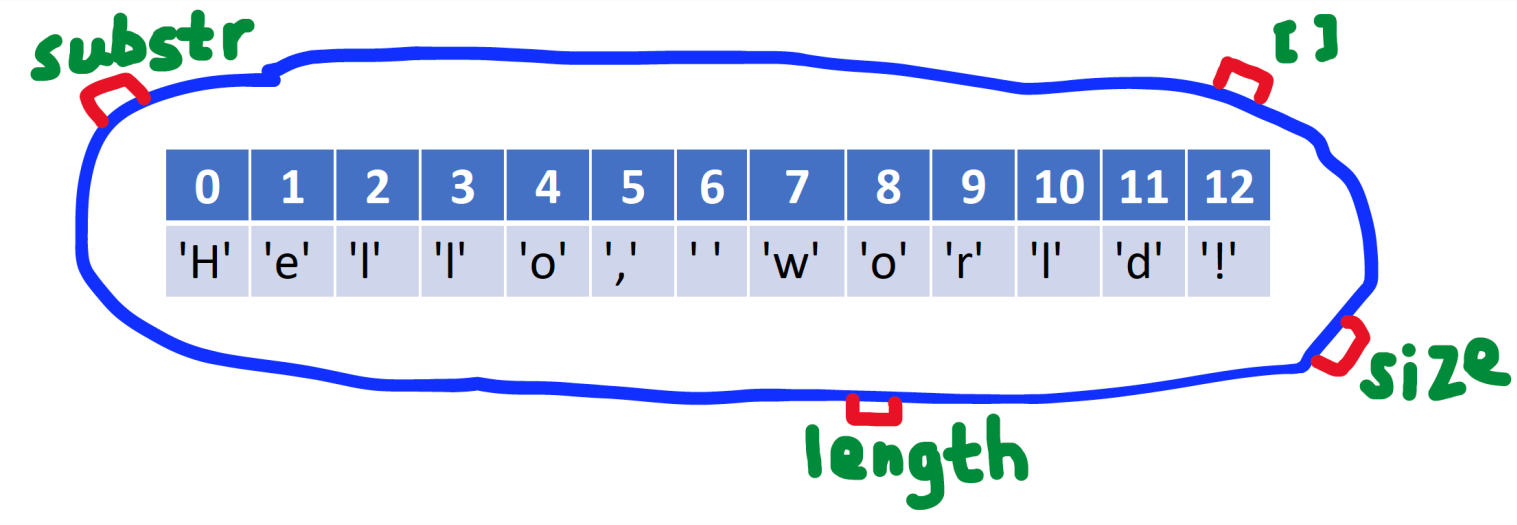
```
std::string greeting = "Hello";
```

Related: Suppose Person class is defined. Declare and instantiate a Person object `ann` with name `"Ann Smith"` and age 18.

```
Person ann("Ann Smith", 18); ✓
Person bob = ("Bob Traylor", 26); ✗
std::ifstream fin("data.txt"); ✓
std::ifstream inp = "data2.txt"; ✗
```

String is a Class

A string is a class encapsulating data (a sequence of characters) and operations on those data.



0	1	2	3	4	5	6	7	8	9	10	11	12
'H'	'e'	'l'	'l'	'o'	','	' '	'w'	'o'	'r'	'l'	'd'	'!'

- length
- size
- concatenation operator +
- []
- at
- find
- substr

Methods of String

Once a string is declared and initialized, we can call its methods using `string_object.method_of_string(proper parameters)`.
If there is no parameter, use an empty pair of parentheses ().
Both `length` and `size` methods of string returns the length of the string, in terms of bytes.

```
size_t length() const;
```

```
size_t size() const;
```

For `std::string`, `length()` and `size()` are interchangeable.
code link `https://onlinegdb.com/ABwQgZNzT`

```
std::string str("hello");
std::cout << "length of string \"" << str << "\" is "
   << str.length() << '\n';
```

concatenation operator +

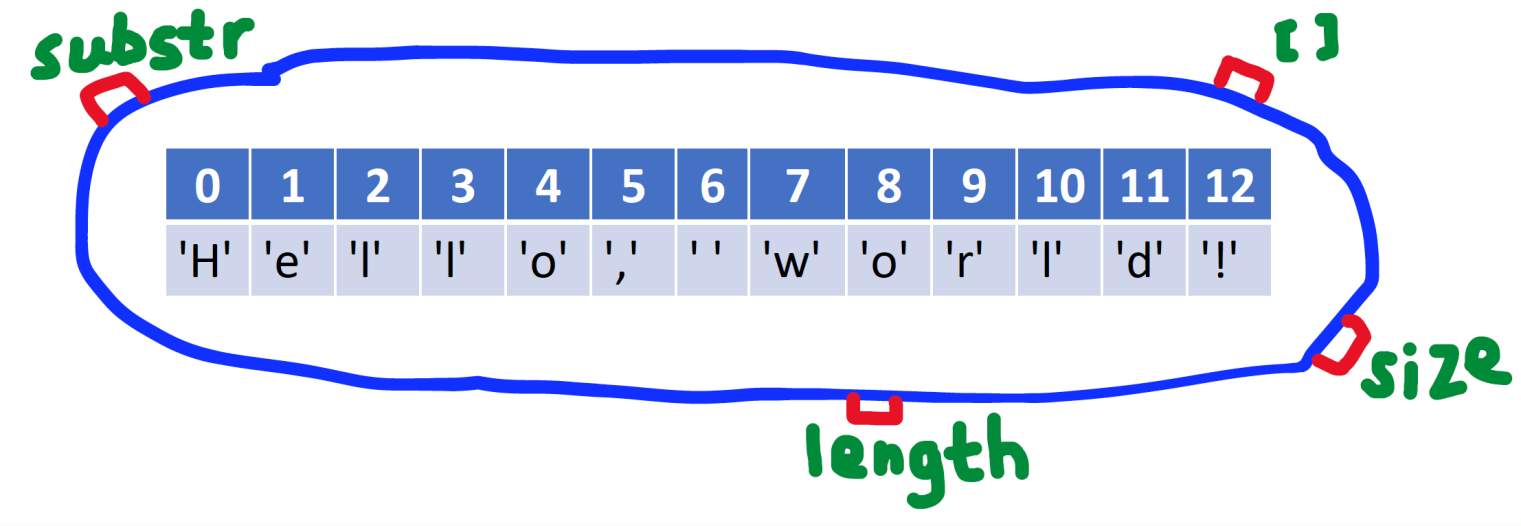
```
str += " world"; //same as str = str + " world";
//(1) + in str + " world" means concatenation,
//    which add the second operand " world"
//    (there is a space character before world)
//    to the end of the first operand str.
//(2) Then put the return of str + " world" to str.
//(3) str is changed to "hello world".

std::cout << "length of string \"" << str << "\" is "
   << str.length() << std::endl;
```

Warning: `"hello" + " world"` does not work.
Reason: `"hello"` and `" world"` are C (language)-style string, which is a sequence of characters – an array of char type – ended by a null character (`'\0'`).
Concatenation operator + does not apply to C-style string.

Index operator []

- Let `str` be a string with values `"Hello world"`.



- Index operator [] returns the element of an array or a string at that index.
  - `str[0]` returns `'H'`. `str[1]` returns `'e'`.
- Warning: index operator [] for a string or an array cannot be written as ().
  - str[0] ✓
  - str(0) ✗
  - Reason: str(0) means to call function str using parameter 0. However, str is not a function name.

Method at

- `char& at (size_t pos);` method access a character at a specified position within the string. It provides bounds checking, meaning it verifies if the given index is within the valid range of the string.
- Let `str` be a string with values `"Hello world"`.

```
std::string str("Hello, world");
```

- `str.at(0)` returns `'H'`. Call method `at` with parameter 0 for object `str`.
- Warning: do not put [] after method name `at`.
  - str.at(0) ✓
  - str.at[0] ✗
  - Reason: `at` is a method of string class. `at` is not an array or a string, so index operator cannot follow `at`.
- Both `str[0]` and `str.at(0)` return the first character of str.
- Prefer to use *stringName*[index] for simplicity.

E4.8

Write a program that reads a word and prints each character of the word on a separate line. For example, if the user provides the input `"Harry"`, the program prints

```
H
a
r
r
y
```

Method find

Search for a string

```
size_t find (const string& str, size_t pos = 0) const;
```

Search for a character

```
size_t find (char c, size_t pos = 0) const;
```

- Type `size_t` is an unsigned int, that is, a non-negative integer.
- Searches the string for the first occurrence of the sequence specified by its arguments.
- When pos is specified, the search only includes characters at or after position pos, ignoring any possible occurrences that include characters before pos.

Method Overload

While a method is technically a function that belongs to a class, the two terms are often used interchangeably.
Two or more functions are overloaded if these functions have

- same name
- similar functionality
- different input parameters (in type, number, and/or order)
- Returns do not matter, they can be the same or different.

Method overloading is a feature of object-oriented programming. Here are examples:

```
size_t find (const string& str, size_t pos = 0) const;
```

```
size_t find (char c, size_t pos = 0) const;
```

The + operator is overloaded. It performs addition when used with numbers, and concatenation when used with strings.

Method Overloading Example

What are the differences between the inputs of home washing machine and coin-operated washing machine?

home washing machine

coin-operated washing machine

find Method Example

Code link is https://onlinegdb.com/cP3jpL-gPu.

```
#include <iostream>     // std::cout
#include <string>       // std::string

int main () {
  std::string str("a cat Cat dogcat");
  std::string str2("cat");

  // different member versions of find in the same order as above:
  std::size_t found = str.find(str2);
  if (found != std::string::npos)
    std::cout << "first \"cat\" found at: " << found << '\n';
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
'a'	' '	'c'	'a'	't'	' '	'C'	'a'	't'	' '	'd'	'o'	'g'	'c'	'a'	't'

find Method Example: II

```
13    found = str.find("cat", found+1);
14    if (found != std::string::npos)
15      std::cout << "second \"cat\" found at: " << found
          << '\n';
16
17    found = str.find(' ');
18    if (found != std::string::npos)
19      std::cout << "space found at: " << found << '\n';
20
21    found = str.find("Dog");
22    if (found == std::string::npos)
23      std::cout << "\"Dog\" not found" << '\n';
24    return 0;
25  }
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
'a'	' '	'c'	'a'	't'	' '	'C'	'a'	't'	' '	'd'	'o'	'g'	'c'	'a'	't'

substr Method

```
std::string substr(size_type pos = 0, size_type len =
    npos) const;
```

- pos: An optional parameter of type size_type (usually std::size_t) representing the starting index of the substring within the original string. The first character is at index 0.
- len: An optional parameter of type size_type representing the number of characters to include in the substring, starting from pos. If pos + len exceeds the string's length, the substring will extend only to the end of the original string. The default value is std::string::npos, which indicates that the substring should include all characters from pos to the end of the string.

code link https://onlinegdb.com/9dZuOxxmI

```
std::string str("Hello, world!");
std::cout << str.substr(7, 3) << "\n";
std::cout << str.substr(5) << "\n";
```

Exercise

1. Enter a name in the format first name first, last name last, then extract the first name and last name.
2. Construct the name in the format last name first and first name last.
3. Print the name in the format of last name first and first name last.
4. For example, suppose you enter enter a name as "Ann Smith", the string you get should be "Smith,Ann".
   Sample input/output

```
Enter a name in first name first, last name last:
    Benjamin Franklin
name in the format of last name first: Franklin,
   Benjamin
```

5. To save time, you only need to define main function, no need to include libraries or write statement to use std (standard) namespace.